BigData Spark Hands-On 2 [part 1]

Spark read/write, Csv, Json

arnaud.nauwynck@gmail.com

Esilv 2025

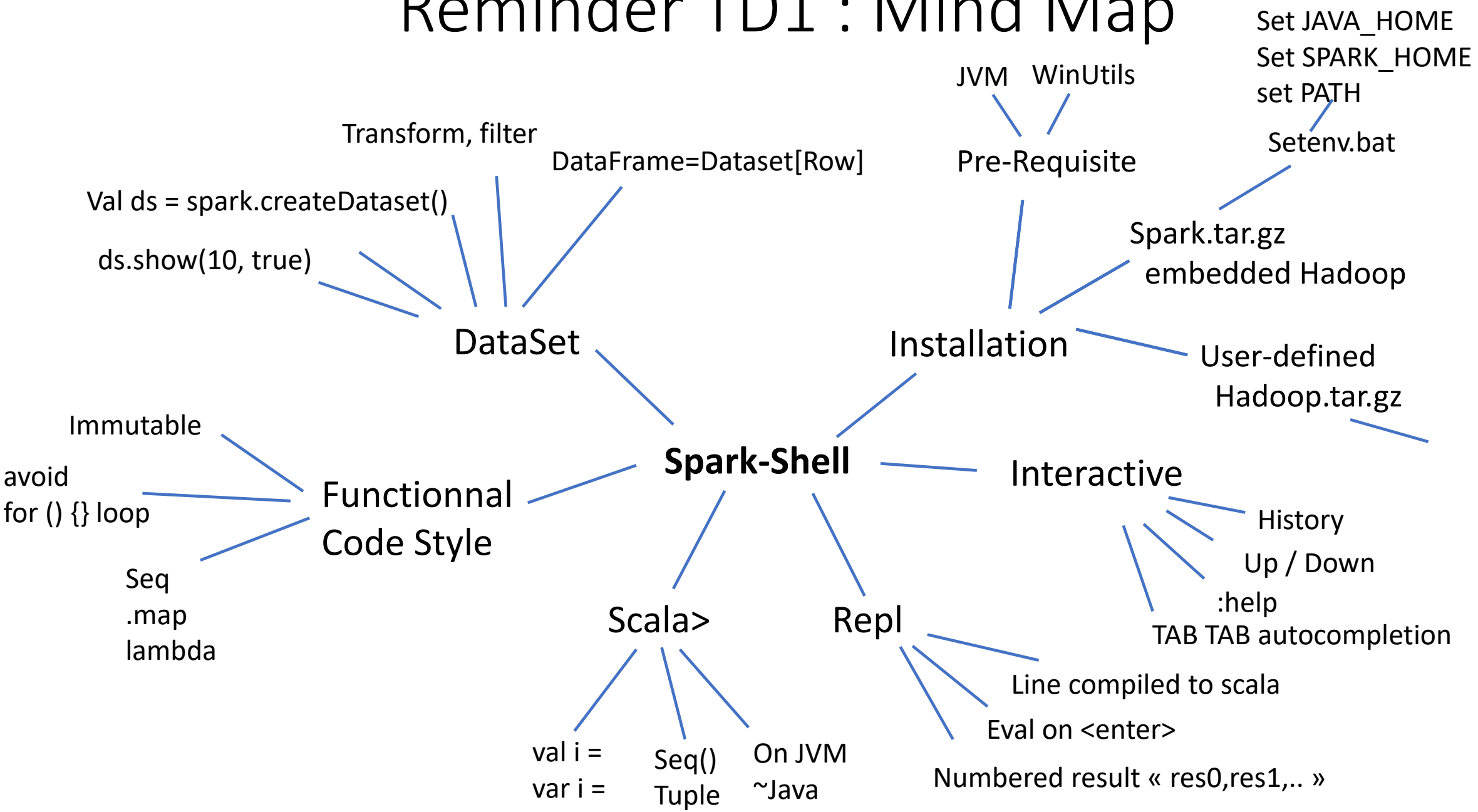
Outline

Reminder on TD1
Questions ? Problems on TD1 ?

Objective of TD2: mostly Files Input-Output

Pre-requisistes: same as TD1: « spark-shell »

Reminder TD1 : Mind Map



Objectives of Hands-On

1/ read CSV files, convert options

2/ read combine several files

3/ others file format: json, nd-json, xml, text

4/ write to PARQUET files dir
repartitionning

5/ Discover PARQUET format optimisations

Pre-Requisites : installation of Spark

From TD1, you have already :

- downloaded + unzipped spark
  In c:\apps\spark\spark-4.0.1
- downloaded + installed java
- downloaded 2 windows specific files for hadoop (winutils.exe + hadoop.dll)

- configured your environment variables
  (SPARK_HOME, HADOOP_HOME, JAVA_HOME, PATH)

- launched « cmd » terminal, and typed « spark-shell »

Running spark-shell
(with more than minimal « default » memory)

spark-shell --driver-memory 10g

Alternative: edit default in file %SPARK_HOME%/conf/spark-defaults.conf
spark.driver.memory 5g

=> PROS: you won't forget each time to pass this parameter

If you don’t increase memory…

For small exercices (1, 2, … 5) : OK, it will work fine
BUT from exercices 6, 7, 8 … it will fail

- Be extremely slow (« swapping » memory to disk)
- Doing Java Full GC (consuming CPU trying to free memory..)
- Then fail with « OutOfMemoryError »

If you have very small PC, with only ~8g of RAM (default should be 16g or 32g ?)
   try allocating ~50g to spark
OR
   work on a smaller file(s) extraction

Exercise 1: edit a csv file with few lines + Header

Get File « sample-data1.csv » from Devinci Online TD2

OR

Edit a CSV file, with columns « id, text, boolean, date, doubleValue, .. »

First line should be « header » : columns names
All other lines = values, separated by « ; »
Strings may contain special ‘;’ character, but should be wrapped with double quotes

Save as file « c:\data\sample-data1.csv »

Exercise 1..

```
id;text;boolean;date;doubleValue
1;hello 1;true;2020/01/24;3.1415926
2;hello 2;false;2020/01/25;2.71828
3;hello 3;true;2020/01/26;1.4142135
4;"hello multi-lines\n line 2..\n line 3";false;2020/01/27;0.0
5;"hello line containing ;;; chars";false;2020/01/28;1.0
```

Exercise 2 : Read CSV file from spark-shell

val **sample1Ds** = spark.read. ….

use
  spark**.read .option**(…) **.format**(« csv ») **.load** (path)
OR equivalent
  spark**.read .option**(…) **.csv**(path)

CHECK :
1/ correct path
2/ option to set the field delimiter
3/ option to interpret the first header line correctly
4/ cast or interpret types correctly
5/ use sample1Ds.show(20, false) to check lines
6/ use sample1Ds.printSchema to check types
  … string,boolean,int should be OK
  for date .. Cf Exercise 3 or 5 !

Exercise 3: use .schema(« col1 type1, ..»)

Add

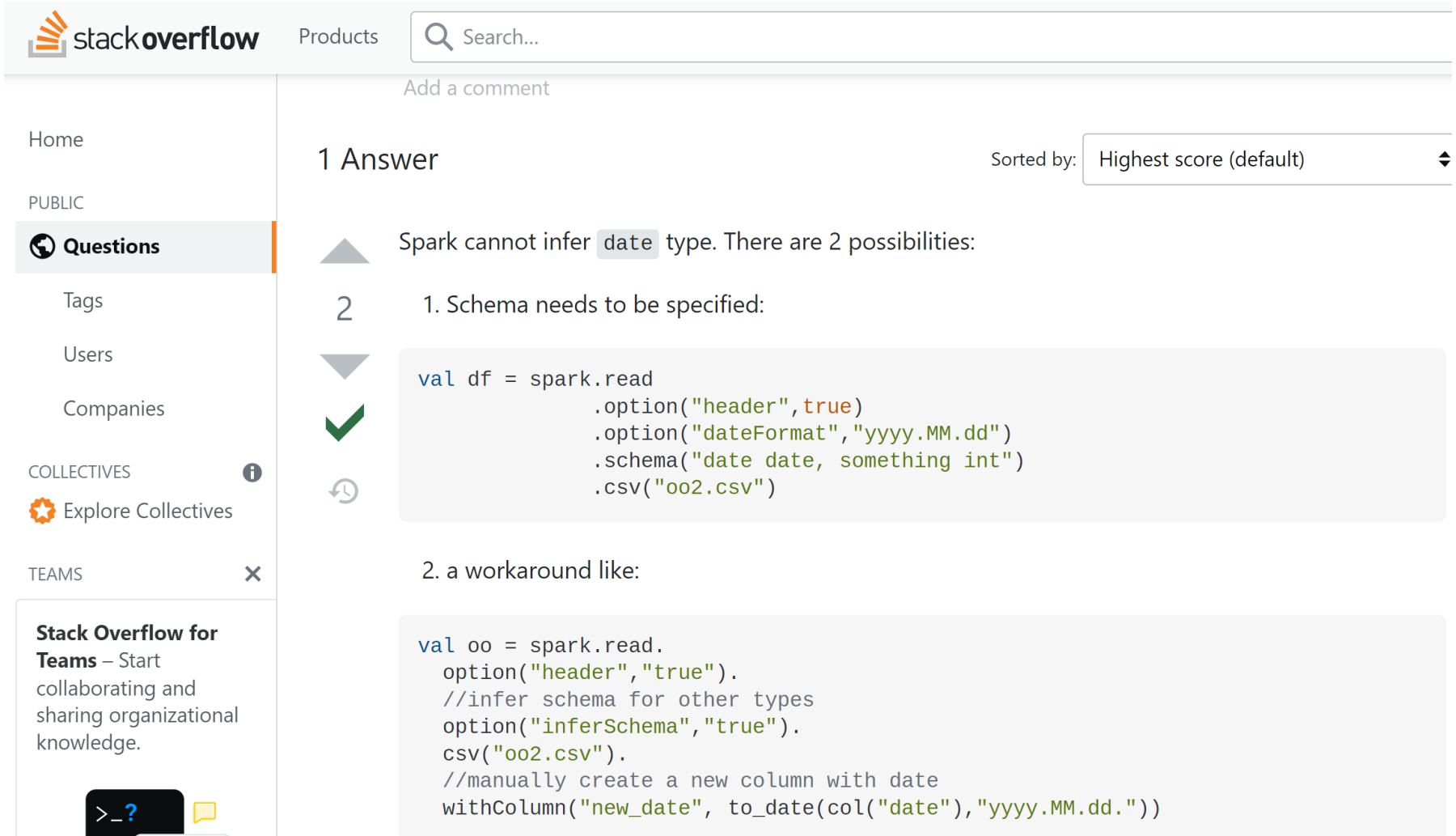
spark.read .. **.schema(** « name1 type1, name2 type2….. »**)**

to force column types

(but maybe not date columns that may need explicit parsing .. cf next !)

Exercise 3: Spark can not infer format for date … expecting yyyy-MM-dd, got yyyy/MM/dd

https://stackoverflow.com/questions/66935214/spark-reading-csv-with-specified-date-format



Exercise 4: use schema for file with no Header line

copy file « sample-data1.csv » without the header line
save as « sample-data1-no-header.csv »

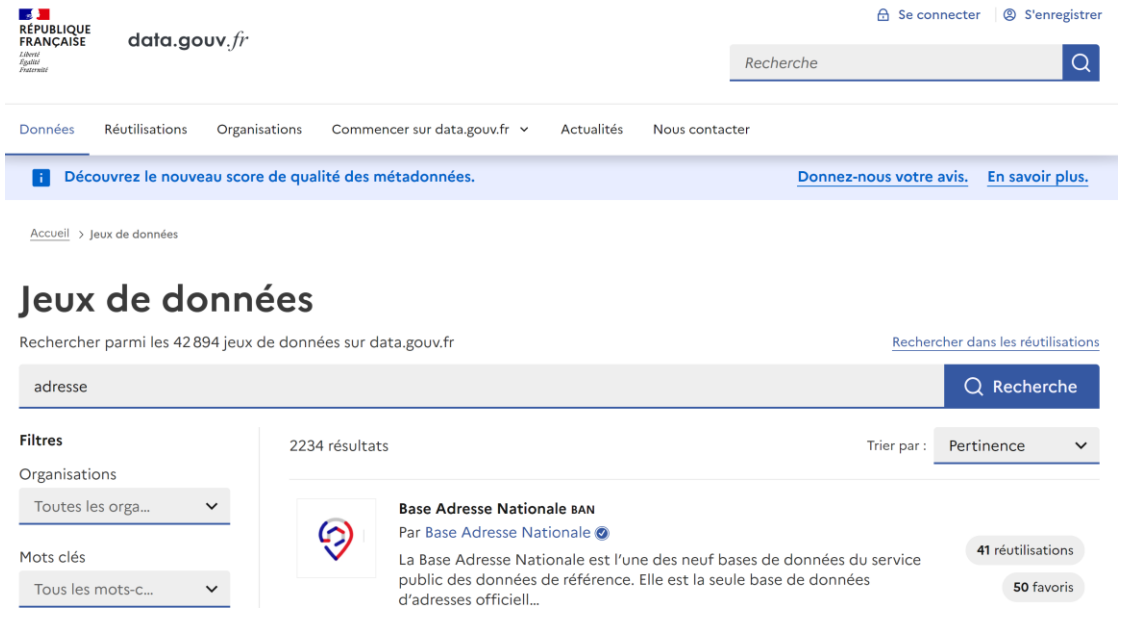
Read CSV file using option(« schema », « ….. »)

Exercise 5: post processing conversion String->Date

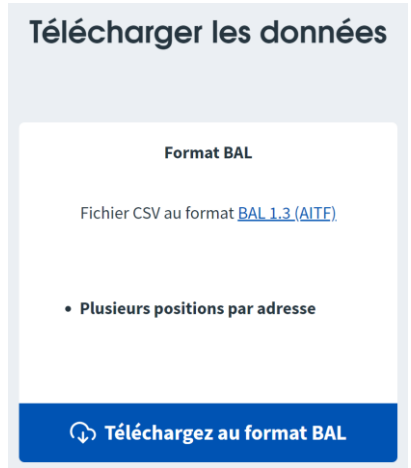
From Dataset sample1Ds (from exercise3/)
… with date as String

Define new column « parsed_date » with typed date,
then drop previous « date » text column,
then rename column

Exercise 6: real CSV Files, example of Open Data







https://www.data.gouv.fr

https://www.data.gouv.fr/fr/datasets/base-adresse-nationale/

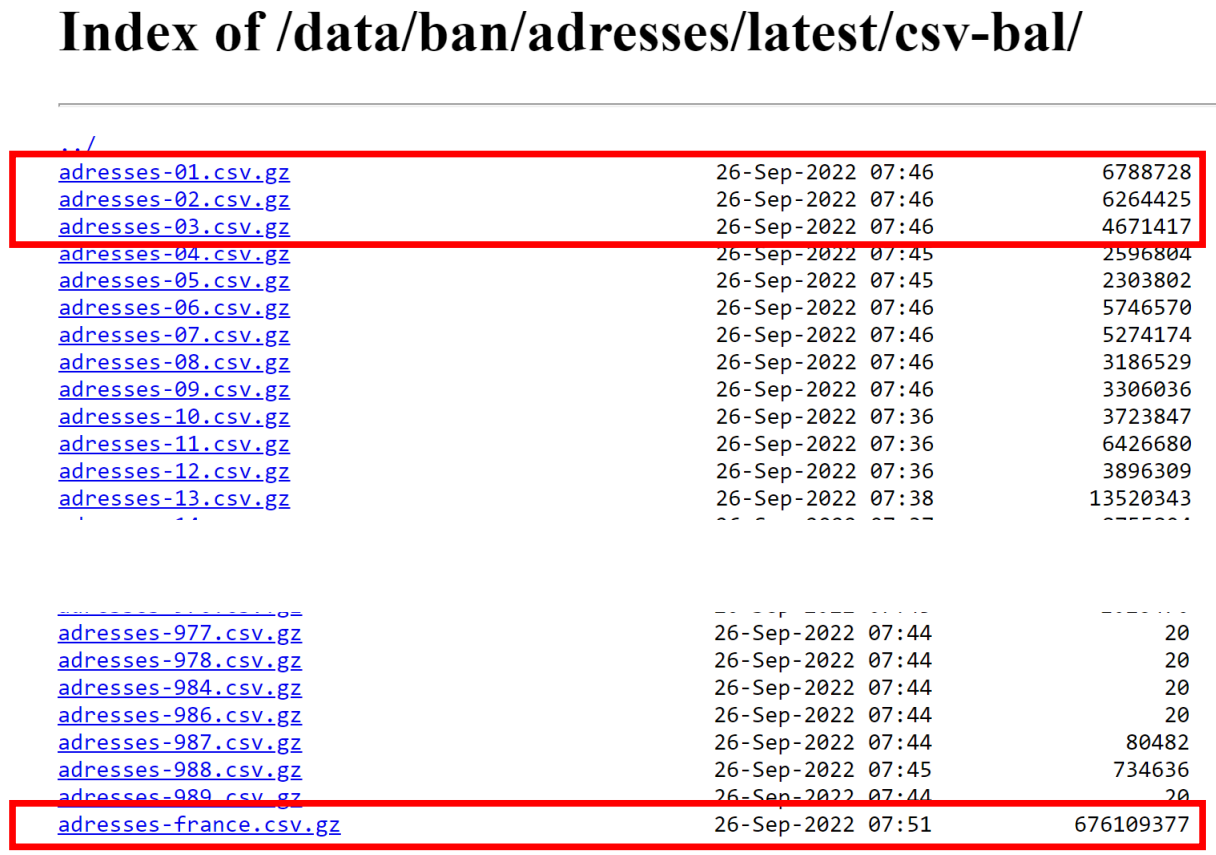
https://adresse.data.gouv.fr/data/ban/adresses/latest/csv-bal

Base Addresse Nationale:
number, street, city, zipCode, latitude, longitude …

1 CSV file 3.3 Go
.. Or split by 990 files x 33 Ko

Exercise 6 … download + uncompress files

https://adresse.data.gouv.fr/data/ban/adresses/latest/csv-bal/



Index of /data/ban/adresses/latest/csv-bal/

../		
adresses-01.csv.gz	26-Sep-2022 07:46	6788728
adresses-02.csv.gz	26-Sep-2022 07:46	6264425
adresses-03.csv.gz	26-Sep-2022 07:46	4671417
adresses-04.csv.gz	26-Sep-2022 07:45	2596804
adresses-05.csv.gz	26-Sep-2022 07:45	2303802
adresses-06.csv.gz	26-Sep-2022 07:46	5746570
adresses-07.csv.gz	26-Sep-2022 07:46	5274174
adresses-08.csv.gz	26-Sep-2022 07:46	3186529
adresses-09.csv.gz	26-Sep-2022 07:46	3306036
adresses-10.csv.gz	26-Sep-2022 07:36	3723847
adresses-11.csv.gz	26-Sep-2022 07:36	6426680
adresses-12.csv.gz	26-Sep-2022 07:36	3896309
adresses-13.csv.gz	26-Sep-2022 07:38	13520343
adresses-977.csv.gz	26-Sep-2022 07:44	20
adresses-978.csv.gz	26-Sep-2022 07:44	20
adresses-984.csv.gz	26-Sep-2022 07:44	20
adresses-986.csv.gz	26-Sep-2022 07:44	20
adresses-987.csv.gz	26-Sep-2022 07:44	80482
adresses-988.csv.gz	26-Sep-2022 07:45	734636
adresses-989.csv.gz	26-Sep-2022 07:44	20
adresses-france.csv.gz	26-Sep-2022 07:51	676109377

Download first 3 files
+
Last (Big) File

990 x small csv.gz files

ALL in ONE = Big csv.gz file

Exercise 6: load multiple CSV files

a/ create a new directory « c:/data/bal-123 »
move the 3 downloaded files « address-{01,02,03}.csv » into it

b/ load datasets from files: (very naive way, not recommended, cf exercise 7/)
address-01.csv -> in dataset01
address-02.csv -> in dataset02
address-03 .csv -> in dataset03

c/ do union of datasets: dataset01**.union**(dataset02)**.union**(dataset03)

check that count of union = sum of count of individual files

Exercise 7: load multiple CSV files from 1 directory

a/ read all files using 1 command: spark.read … taking path of directory (instead of path of files)

b/ check that count is similar to exercise 6/

c/ create another empty directory « c:/data/ball-all-uncompressed/ »,
and put the Big downloaded file « address-france.csv », uncompressed from the « csv.gz »
… load it

d/ create another empty directory « c:/data/ball-all-gz/ »,
and put the Big downloaded file « address-france.csv.gz »
… load it

e/ interpret slowness of d/, compare to fast speed of c/, and to difficulty of ex 6/ for 990 files

Exercise 8 … repeat query before/after .cache() explain which query a/b/c/.. perform IO reads

scala> spark.read. … « infer »true ⟵ a/ Read? … for schema/statistics, but data not retained in memory

scala> allAdressCsvDs.count ⟵ b/ Read ?

scala> allAdressCsvDs.count ⟵ c/ Read ?

scala> allAdressCsvDs.**cache()** ⟵ d/ Read ?

scala> allAdressCsvDs.count ⟵ e/ Read ?

scala> allAdressCsvDs.count ⟵ f/ Read ?

Objectives of Hands-On



1/ read CSV files, convert options

2/ read combine several files

3/ others file format: json, nd-json, xml, text

4/ write to PARQUET files dir
repartitionning

5/ Discover PARQUET format optimisations

10 mn pause

APPENDIX …
If you have finished exercises 1-8, but your friends are still struggling ?

Idea 1 / help your struggling friends

Idea 2/ imagine processing queries on OpenData addresses
(see next)

APPENDIX 1: Exercise 6 was loading « addresses » …

What could you process out of this ?

Suggestions:

1. Extract all city names with zipCode, average latitude/longitude
2. Extract all distinct street names
3. Extract number of addresses and number of streets per city
4. Detect all street named to a person « firstname Lastname » given a list of known firstname
5. Count how many times « Victor Hugo » is present
6. Count cities having neither « Victor Hugo », nor « Balzac », or « Picasso »
7. Detect cities having one of « Leningrad », « Stalingrad », « Maurice Thorez »
8. …

Spoiler alert (for NEXT TD 3) …
SQL Table = Directory

SQL API ←same as→ Dataset IO API

```
ds = spark.sql(
        « SELECT * FROM mytable »)
```

```
ds = spark.read.option(…)
        .load(« my_directory »)
```

SELECT Sql Table ←same as→ Read Directory
(scan all *.* files)

ds.saveAsTable()
spark.sql(« INSERT … INTO table ») ←same as→ Overwrite all files on directory

To Remember

You don’t read a specific file name, but you read *.* from a Directory

You don’t write a specific file (it generates many files, with UUID names), in a directory

To be atomic, spark write to « _temporary » sub dir, then delete all old files and move new files

You can’t update a specific file or row in a file … you delete the whole directory, and replace with new files

There is no sql « UPDATE » built-in in spark   (see next for « Iceberg » plugins and new file formats)

Search & Try yourself

```
//val ds =    spark.read. … csv()

ds.saveAsTable(« yourTableName »)

spark.sql(« select * from yourTableName »).show(5, false)

OR

ds.registerTempView(« yourView1 »)

spark.sql(« select * from yourView1 »).show(5, false)
```

Objectives of Hands-On



1/ read CSV files, convert options

2/ read combine several files

3/ others file format: json, nd-json, xml, text

4/ write to PARQUET files dir
repartitionning

5/ Discover PARQUET optimisations

6/ Discover Spark UI

Exercise 9: save as json, load json file

a/ write addresses Dataset (from CSV) to JSON

b/ look at written files
  is it a valid « .json » file, containing 1 array of objects ?
  Or ND-json = New-lines Delimited Json

  compare file size of csv and nd-json

c/ load dataset from ND-json file

Exercise 10 : analyze written json files

Repeat the same operation twice, and notice that

1/ you can not specify the file names .. Only an empty directory name !

2/ the file names are generated with a unique UUID
part-00{partNumber}-{UUID}-c000.json

3/ during write time… spark write to « _temporary » sub dir, then rename at the end

4/ spark has written several files …
If using the full dataset of 3Giga .. Spark write 26 json files

(Optional) Exercise : convert ND-json to valid JSON array File
… then load from spark

To convert … you need to

1. add a START_ARRAY delimiter at line 1 : char « **[**«
2. finish all lines with char « **,** » … except the last line !!
3. finish END_ARRAY at last line : char « **]** »

a/ edit manually a little file (~10 lines) / use regexp / or write a program(?)

b/ load this real json « multiline » file from spark

Exercise 11 : ND-JSON with nested struct fields (sub-objects structs, arrays, map)

a/ Save the following ND-JSON text to File « c:/data/sample-json-abcd/sample.json »

```
{"a": "hello" }
{"b": "hello" }
{"c": { "d": [ 1, 2, 3 ],  "pts": [  {"x":1, "y":1}, {"x":2, "y":1}, {"x":1, "y":2} ] } }
```

b/ load the directory, using

```
val dsJson = spark.read.json("C:/data/sample-json-abcd")
```

c/ display the « schema », using dsJson.printSchema()
interpret

d/ redo load without the implicit « inferSchema », by forcing a schema that only

Optionnal Exercise 11 bis:
force a « spark.read.schema(...) » (avoid infer)

```
import org.apache.spark.sql.types._
val schema = StructType( … ommitted code …. for « a », and « c », but not « b »….   )
    OR
val schema = « a string; c : STRUCT< …. Ommitted code > « ;

// Read JSON with forced schema
val dsJson = spark.read.schema(schema).json("C:/data/sample-json-abcd")

dsJson.printSchema()
dsJson.show(false)
```

a/ fill missing code for schema=.., then execute above code
b/ check that field « b » is not present in schema, nor in « .show() »
c/ but still present in File… interpret

Hint Exercise 11 bis

Field « b »
exists in json file,
but ignored in read



```
scala> val dsJson = spark.read.schema(schema).json("C:/data/sample-json-abcd")
val dsJson: org.apache.spark.sql.DataFrame = [a: string, c: struct<d: array<bi
>>>]

scala>

scala> dsJson.printSchema()
root
 |-- a: string (nullable = true)
 |-- c: struct (nullable = true)
 |    |-- d: array (nullable = true)
 |    |    |-- element: long (containsNull = true)
 |    |-- pts: array (nullable = true)
 |    |    |-- element: struct (containsNull = true)
 |    |    |    |-- x: long (nullable = true)
 |    |    |    |-- y: long (nullable = true)


scala> dsJson.show(false)
+-----+-----------------------------------+
|a    |c                                  |
+-----+-----------------------------------+
|hello|NULL                               |
|NULL |NULL                               |
|NULL |{[1, 2, 3], [{1, 1}, {2, 1}, {1, 2}]}|
+-----+-----------------------------------+
```

(Optional) …. Try using SQL extension « LATERAL VIEW » and « explode() »

```
dsJson.createOrReplaceTempView("dsJsonView");
val flatDs = spark.sql("""
      SELECT a,
            ptsElement.x as x,
            ptsElement.y as y
      FROM dsJsonView
      LATERAL VIEW explode(c.pts) exploded_pts AS ptsElement
    """)
flatDs.show()
```

(Optional) Hint Exercise Lateral View explode

```
scala> dsJson.createOrReplaceTempView("dsJsonView");val flatDs = spark.sql("""
     |         SELECT a,
     |                ptsElement.x as x,
     |                ptsElement.y as y
     |         FROM dsJsonView
     |         LATERAL VIEW explode(c.pts) exploded_pts AS ptsElement
     |       """)
val flatDs: org.apache.spark.sql.DataFrame = [a: string, x: bigint ... 1 more field]

scala> flatDs.show()
+----+---+---+
|   a|  x|  y|
+----+---+---+
|NULL|  1|  1|
|NULL|  2|  1|
|NULL|  1|  2|
+----+---+---+


scala> flatDs.printSchema()
root
 |-- a: string (nullable = true)
 |-- x: long (nullable = true)
 |-- y: long (nullable = true)
```

Exercise 12 : Read plain « Text » lines from file

a/ edit a file containing many lines of text

b/ load this text file from spark

c/ count number of lines

d/ what is the difference between spark.read.textFile() and spark.read.text() ?

Exercise 13 : What about xml format ?

In spark version <= 3.5 …. There was NO « xml() » method to support file !?
  (required databricks-xml plugin)
what ’s new in spark 4.0 ? … xml (and spark-connect !)

a/ create a new directory « c:/data/sample-xml/»
 … and put file downloaded from Devinci Online « sample1.xml »

b/ try loading Dataset for « c:/data/sample-xml »

```
val dsXml = spark.read.format("xml") ….  truncated code, to search
```

c/ check dsXml.show() and dsXml.printSchema()

Exercise 13 file « sample1.xml »

```
<root>
 <element attribute1="abc">
        <a>Hello</a>
        <b>123</b>
        <pts>
                <pt x="1" y="1"/>
                <pt x="2" y="2"/>
        </pts>
 </element>
 <element attribute1="def">
        <a>World</a>
        <b>234</b>
        <pts>
                <pt x="1" y="1"/>
                <pt x="2" y="2"/>
        </pts>
 </element>
</root>
```

Exercise 13 …

```
scala> val dsXml = spark.read.format("xml").option("rowTag", "element").load("C:/data/sample-xml")
val dsXml: org.apache.spark.sql.DataFrame = [_attribute1: string, a: string ... 2 more fields]

scala> dsXml.printSchema()
root
 |-- _attribute1: string (nullable = true)
 |-- a: string (nullable = true)
 |-- b: long (nullable = true)
 |-- pts: struct (nullable = true)
 |    |-- pt: array (nullable = true)
 |    |    |-- element: struct (containsNull = true)
 |    |    |    |-- _x: long (nullable = true)
 |    |    |    |-- _y: long (nullable = true)


scala> dsXml.show()
+-----------+-----+---+-----------------+
|_attribute1|    a|  b|              pts|
+-----------+-----+---+-----------------+
|        abc|Hello|123|{[{1, 1}, {2, 2}]}|
|        def|World|234|{[{1, 1}, {2, 2}]}|
+-----------+-----+---+-----------------+
```

… Example of « Big » Xml dump files

https://dumps.wikimedia.org/enwiki/20220901/

More than 100 Gigas of Xml files

2022-09-02 04:32:53 **done** Articles, templates, media/file descriptions, and primary meta-pages, in multiple bz2 streams, 100 pages per stream

enwiki-20220901-pages-articles-multistream1.xml-p1p41242.bz2 251.2 MB
enwiki-20220901-pages-articles-multistream-index1.txt-p1p41242.bz2 221 KB
enwiki-20220901-pages-articles-multistream2.xml-p41243p151573.bz2 336.5 MB
enwiki-20220901-pages-articles-multistream-index2.txt-p41243p151573.bz2 638 KB
enwiki-20220901-pages-articles-multistream3.xml-p151574p311329.bz2 364.6 MB
enwiki-20220901-pages-articles-multistream-index3.txt-p151574p311329.bz2 820 KB
enwiki-20220901-pages-articles-multistream4.xml-p311330p558391.bz2 406.4 MB
enwiki-20220901-pages-articles-multistream-index4.txt-p311330p558391.bz2 1.3 MB
enwiki-20220901-pages-articles-multistream5.xml-p558392p958045.bz2 436.6 MB

Objectives of Hands-On



1/ read CSV files, convert options

2/ read combine several files

3/ others file format: json, nd-json, xml, text

4/ write to PARQUET files dir
repartitionning

5/ Discover PARQUET format optimisations

APPENDIX …
If you have finished exercises 9-13, but your friends are still struggling ?

Idea 1 / help your struggling friends

Idea 2/ bonus exercise 14 on textFile

Exercise 14 : the « World Count » Hello-World standard program of BigData

From dataset of text lines from exercise 12/

split all lines into words
… For this, start by « map() » replacing all punctuation marks by a single space
… Then replace 2 successive spaces by a single space
… Then split by space

… then use « flatMap() » instead of « map() »

Finally extract words count

10 mn pause

Continue…

Please open next document: Part2
TD2-Part2 (PARQUET)